live university

FÓRUM DE IA
Demonstração Machine Learning

© Live University. This work is licensed under the Creative Commons. If you want to use or share, you must give appropriate credit to Live University.

Demonstração prática

O objetivo desse material é demonstrar um framework bastante utilizado na indústria para projetos envolvendo machine learning , e comparar a performance de três algoritmos (Random Forest Classifier, Support Vecor Machine e K-Nearest Neighbors) bastante utilizados no mercado, abordando alguns conceitos chave de Machine Learning .

Introdução ao People Analytics

People Analytics, ou HR Analytics (Human Resources Analytics) é como é chamado o campo analítico que se refere a aplicação de técnicas analíticas no departamento de recursos humanos de uma empresa com o objetivo de melhorar a performance e satisfação dos funcionários e assim gerar um retorno maior frente ao grande investimento que uma empresa tem que fazer para contratação e treinamento de funcionários.

Problema abordado: Prever a taxa de satisfação dos funcionário

Muitos dos estudos presentes na literatura sobre people analytics utilizam modelagem com machine learning para gerar modelos preditivos que prevejam a probabilidade de saída de um funcionário (taxa de atrito ou ainda rotatividade). Nessa demonstração, vamos mudar o foco: prever a taxa de satisfação do funcionário ao invés da probabilidade de saída dele.

Base de dados utilizada

Para essa demonstração, utilizaremos uma base de dados pública (dados reais porém descaracterizados de uma empresa não divulgada). Essa base é utilizada para criar modelos que prevejam a probabilidade de atrito de um determinado funcionário

Linguagem utilizada na demonstração

Python

Uma das linguagens de programação mais populares no mercado atualmente em Data Analytics e em muitas outras aplicações. Dentre as vantagens de se utilizar o python, duas que pesam muito são:

1) Facilidade: o python é uma linguagem semântica, de fácil aprendizado.
2) Comunidade ativa: o python conta com uma comunidade bastante ativa que além de auxiliar bastante em fóruns de perguntas e tutoriais, também produz e mantém diversas bibliotecas utilizadas nas mais diversas aplicações, inclusive em data analytics.

Embora utilizemos o python para a demonstração, o grande objetivo é exemplificar aplicações de machine learning, e não introduzir o python e sua sintaxe

Importação da base de dados

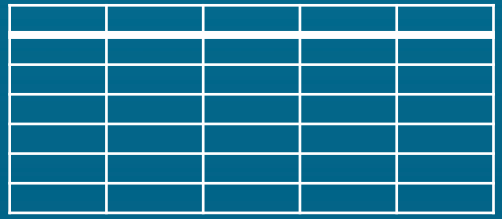
Os dados que usamos na demonstração dados vindos de um arquivo CSV. Essa é apenas uma de inúmeras diferentes fontes de dados que podem ser utilizadas. Hoje em dia, os dados podem vir de fonte internas da empresa, como o CRM, SRM, ERPs em geral, assim como podem ser coletados de fontes externas como redes sociais, medições por IoT, questionários, API's, Web Scrapping, etc.

O fato de existirem muitos tipos de fontes de dados disponíveis também gera um volume e variedade muito grande de dados para serem trabalhados. Geralmente, dados podem ser classificados como Estruturados, Semiestruturados e Não estruturados dado o tipo de estrutura como esses dados são armazenados. Cada tipo e estrutura de dados exige um tratamento e preparação diferente, e isso é um dos grandes motivos de existir complexidade ao se trabalhar com Big Data.



live university

Dados Estruturados



Dados Semiestruturados

< XML >

{ JSON }

< /HTML >

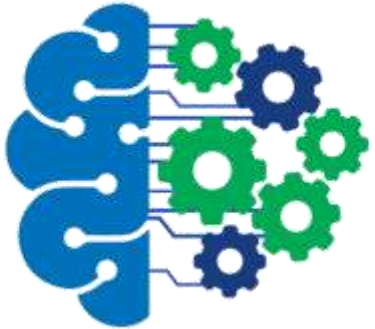
Dados Não Estruturados



Machine learning

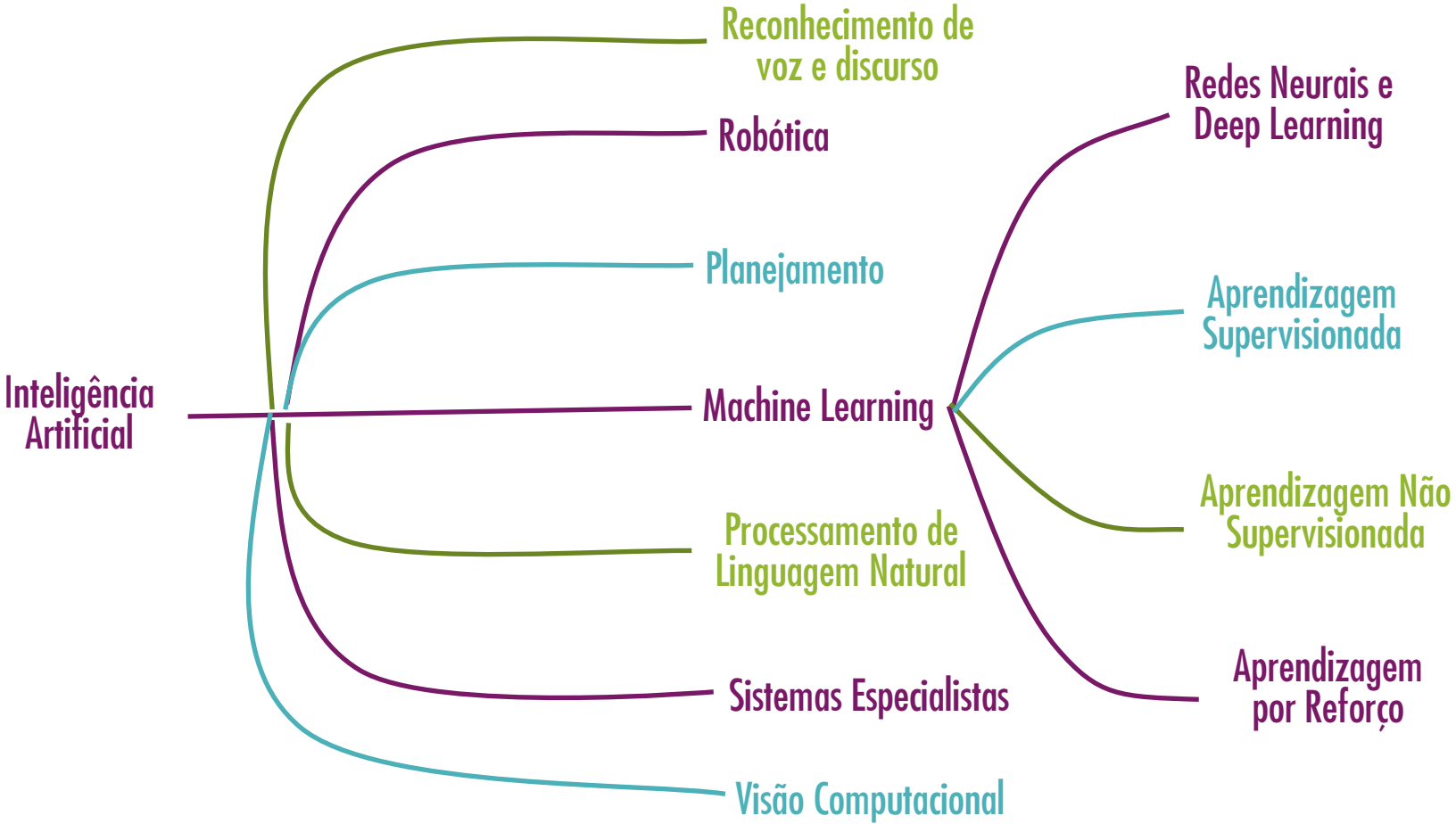
Machine learning (aprendizado de máquina) é um subcampo de inteligência artificial que usa algoritmos que são alimentados com dados, realizam um processo de aprendizagem (chamada fase de treino), e que gera um modelo matemático ajustado aqueles dados. Esse modelo pode então ser utilizado com novos dados para realizar classificações, previsões, segmentações, regressões, etc.

A maneira como o a fase de treinamento é realizada define o tipo de aprendizagem (seja ele supervisionado, semissupervisionado, não supervisionado ou por reforço). Existem algoritmos específicos para cada tipo de aprendizagem.

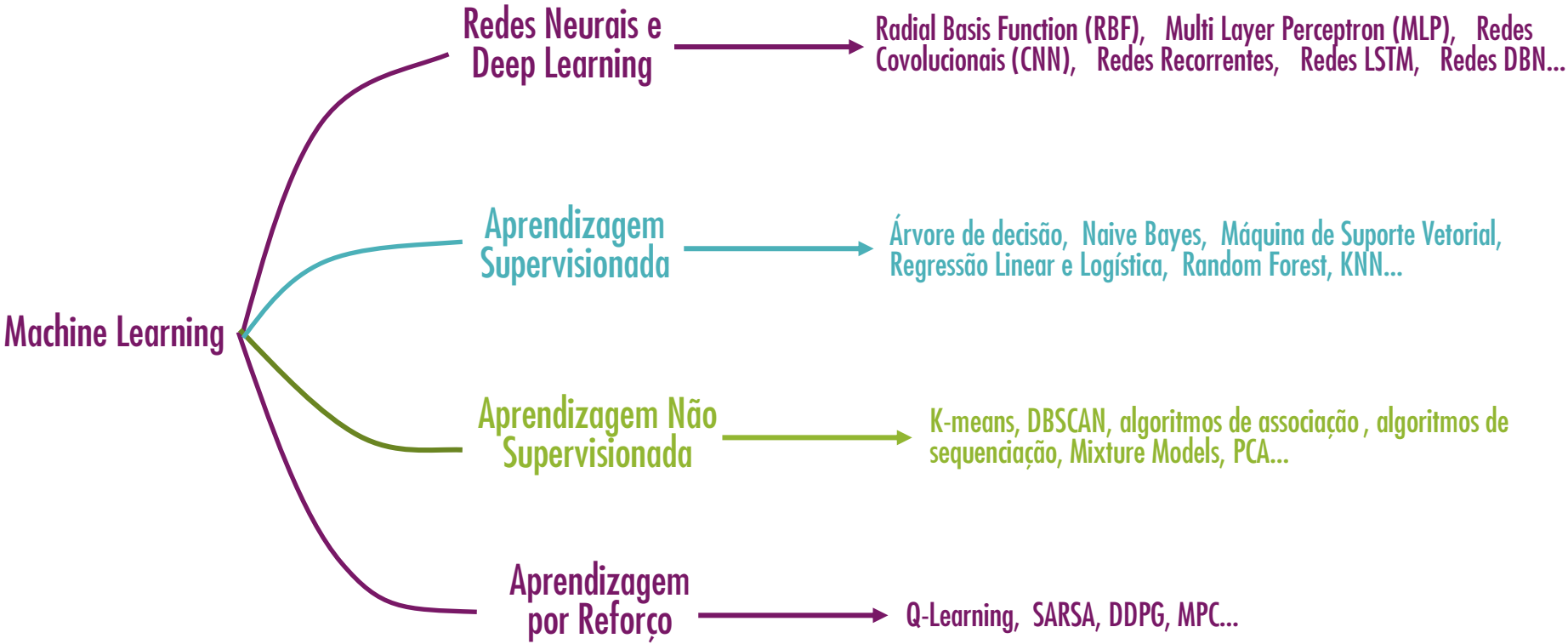


FÓRUM DE IA

live university



live university



Machine learning – Tipo de aprendizagem

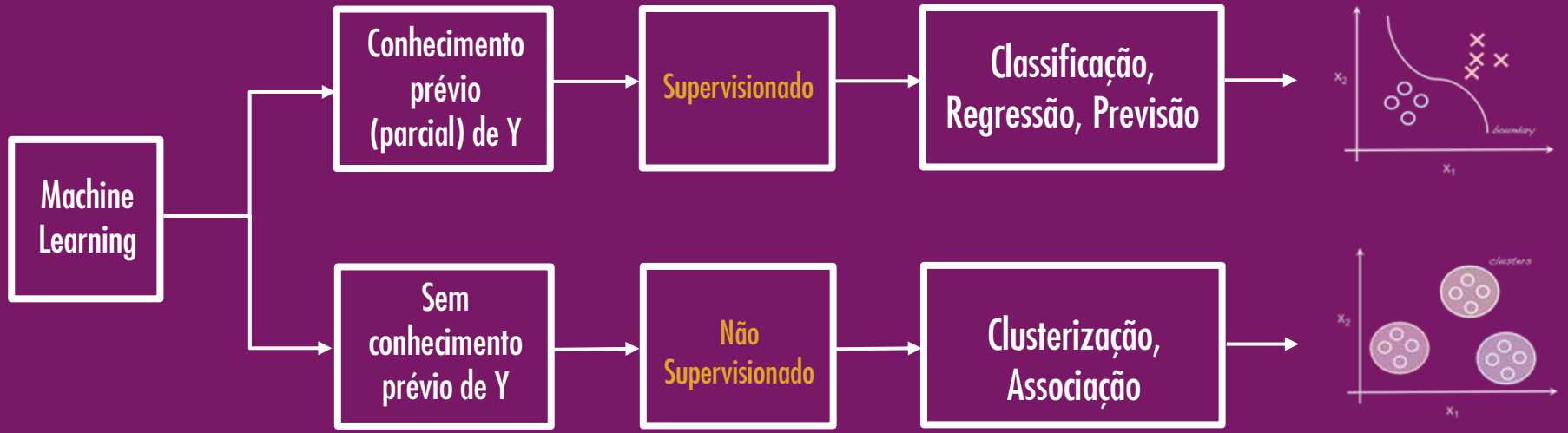
Nessa demonstração usamos dados estruturados e rotulados. Por dados rotulados, significa que queremos criar um modelo que precisa ser capaz de prever ou classificar uma variável já existente no conjunto de dados que usamos para alimentar o algoritmo. Ou seja, o algoritmo precisa entender como as outras variáveis daquele conjunto de dados (chamadas de variáveis independentes) se relacionam e influenciam variável que queremos prever (variável dependente)

Variável independente	Variável independente	Variável independente	Variável independente	Target, Variável Dependente
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y



live university

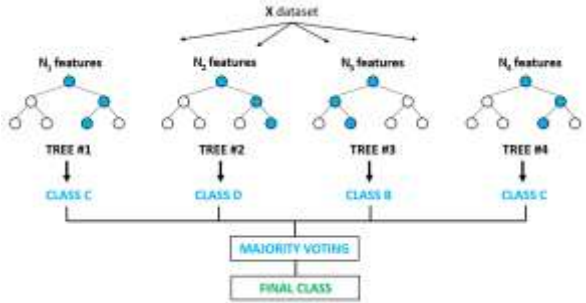
Aprendizagem Supervisionada vs Não Supervisionada



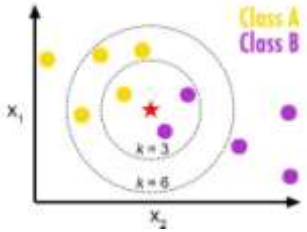
Machine learning - Algoritmos

Usaremos três modelos de aprendizagem supervisionada nessa demonstração, afim de comparar a performance deles e escolher o modelo que melhor se ajusta ao problema: Random Forest Classifier, Support Vecor Machine (Máquina de suporte vetorial) e K-Nearest Neighbors (KNN).

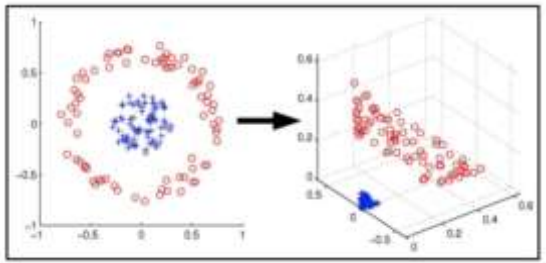
Random Forest Classifier



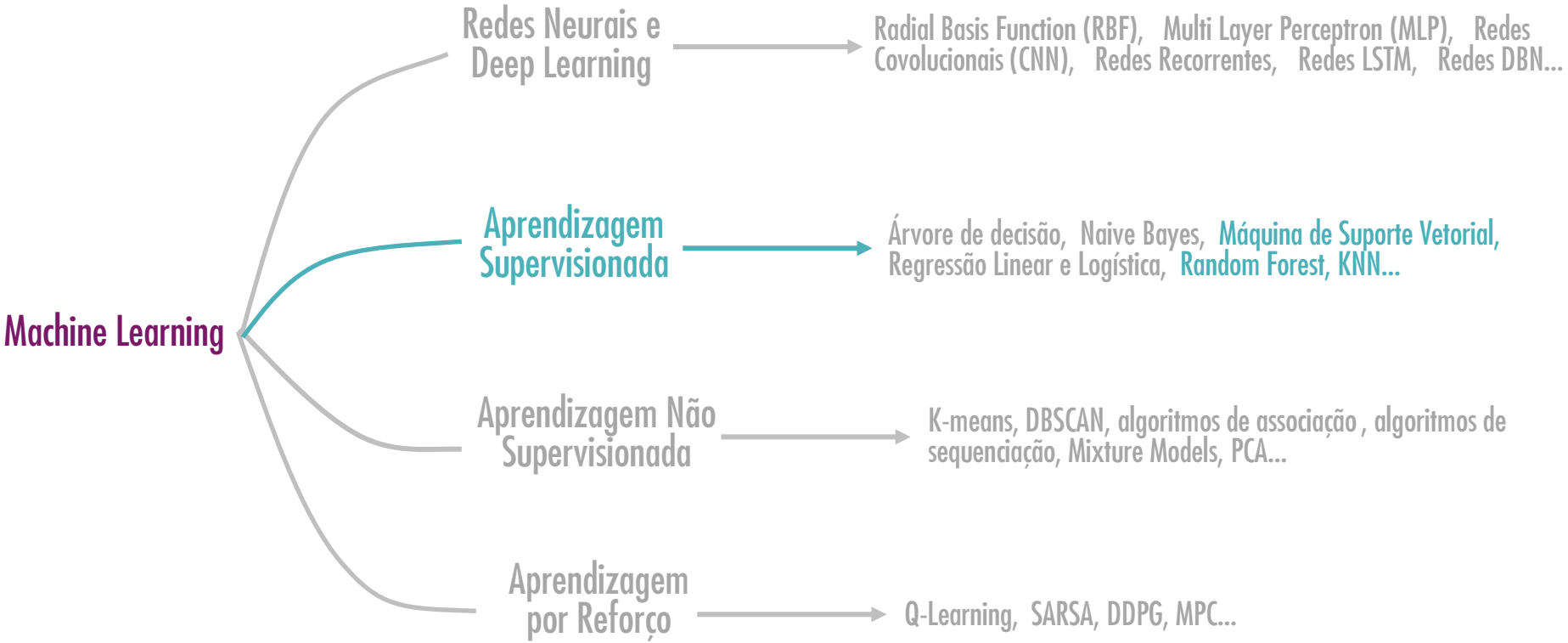
K-Nearest Neighbors (KNN)



Máquina de suporte vetorial (SVC)

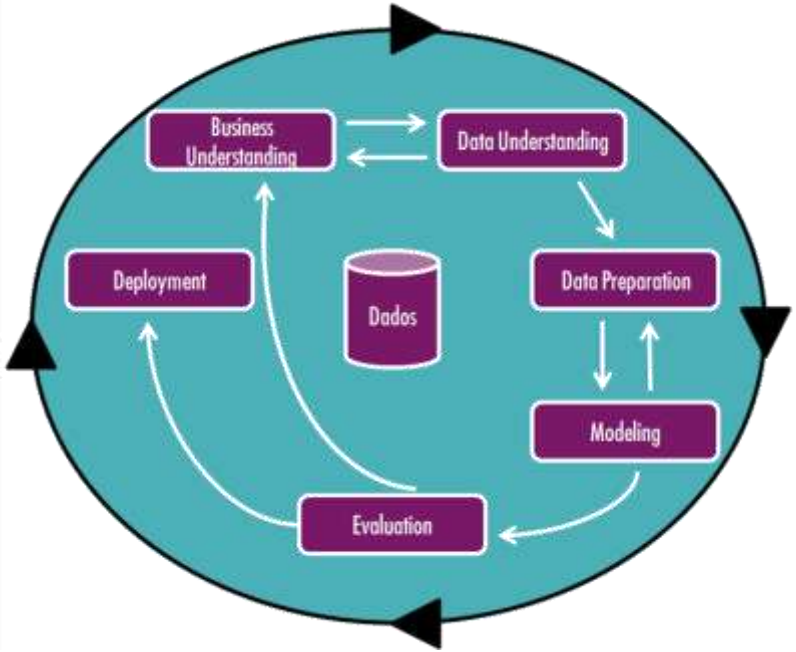


live university



Machine learning - Framework

Para realizar essa demonstração, utilizamos um framework bastante difundido em mineração de dados chamado **CRISP-DM** (cross-industry standard process for data mining)



Business Understanding: Compreender o negócio

Data Understanding: Compreender os dados

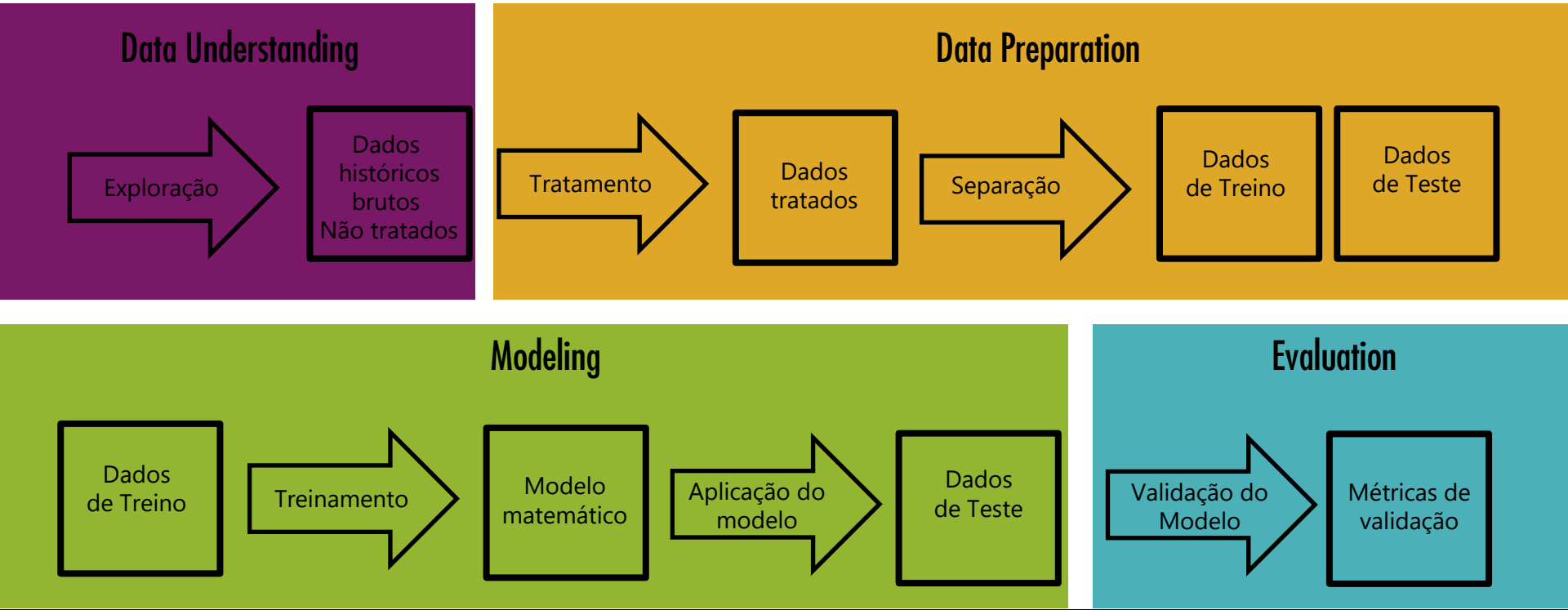
Data Preparation: Preparar os dados

Modeling: Realizar o processo de modelagem, cujo resultado é um modelo matemático

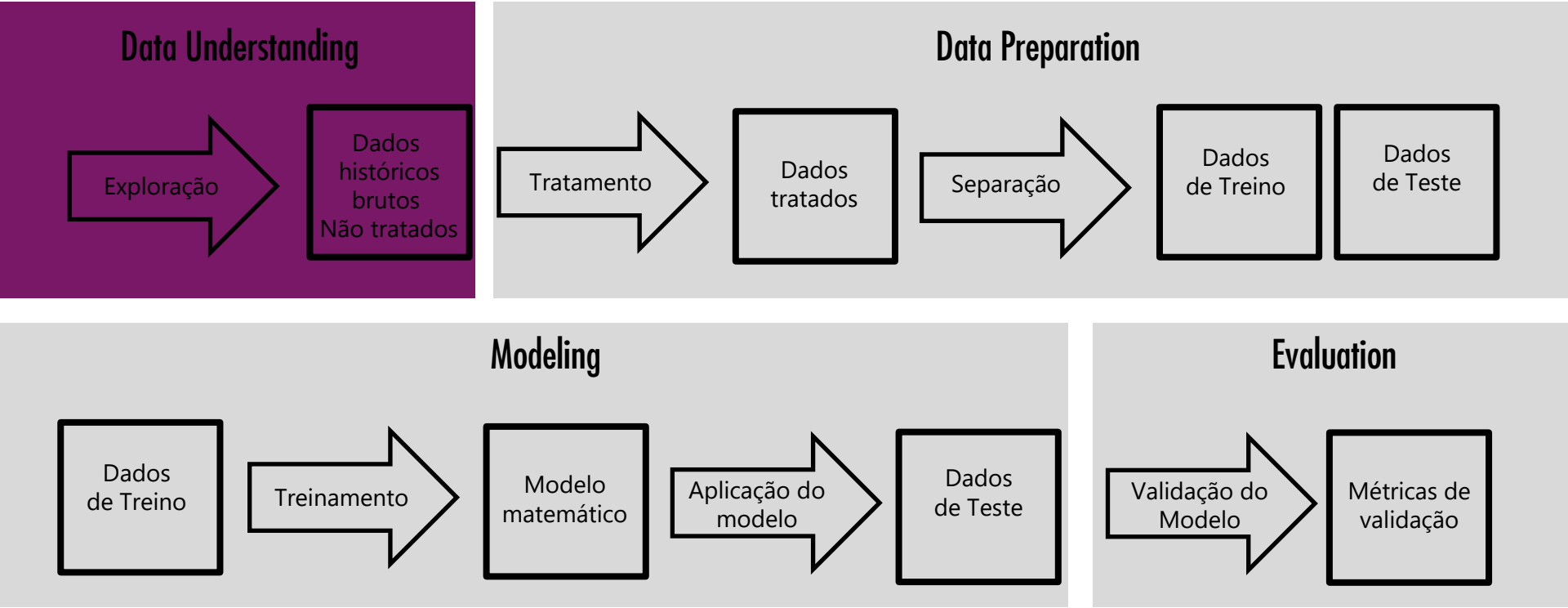
Evaluation: Teste e avaliação da performance do modelo por métricas bem definidas

Deployment: Implementar o modelo para o fim proposto

Para os propósitos desta demonstração, iremos abordar as etapas de Data Understanding até Evaluation:



Para os propósitos desta demonstração, iremos abordar as etapas de Data Understanding até Evaluation:



Data Understanding

Nessa parte, verificamos como os dados foram importados para dentro do ambiente do Python, em especial: Quais as dimensões da base de dados, quais colunas estão presentes, quais podem ser descartadas, qual o tipo de dado de cada coluna (MUITO IMPORTANTE!)

Variável independente	Variável independente	Variável independente	Variável independente	Target, Variável Dependente
A	1	A	100	Y
D		B	122	Y
C	4	B	76	Y
E	3			Y
F	2	A	1300	Y
	2	a	97	Y

Detectar valores nulos e valores anômalos também são práticas fundamentais. Campos vazios podem tornar uma linha inteira do dataset inutilizável dependendo da aplicação, além de afetarem o modelo matemático que é gerado a partir dos dados. Dados anômalos, conhecidos como outliers, assim como os campos vazios, também podem afetar a eficácia do modelo, pois são valores que fogem muito do comportamento considerado normal da maioria dos valores, distorcendo as regras que são extraídas dos dados.

Data Understanding

Os dados foram importados em estruturas tabulares chamadas dataframes. Cada linha representa um funcionário da empresa. Como foi mencionado na introdução, esse dataset (como é chamado o conjunto de dados), é usado para treinar modelos para classificar a taxa de atrito em RH, determinado pela variável (coluna) **left** (1 se o funcionário saiu, 0 se ele não saiu), porém, nossa variável dependente no caso será a taxa de satisfação (**satisfaction_level**). Queremos gerar um modelo que consiga determinar o nível de satisfação de um novo funcionário utilizando as variáveis independentes coletadas dele.

Y	X1	X2	X3	X4	X5	X6	X7	X8

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	Departments	salary
0	0.38	0.53	2	157	3	0	1	0	sales	low
1	0.80	0.86	5	262	6	0	1	0	sales	medium
2	0.11	0.88	7	272	4	0	1	0	sales	medium
3	0.72	0.87	5	223	5	0	1	0	sales	low
4	0.37	0.52	2	159	3	0	1	0	sales	low

Data Understanding

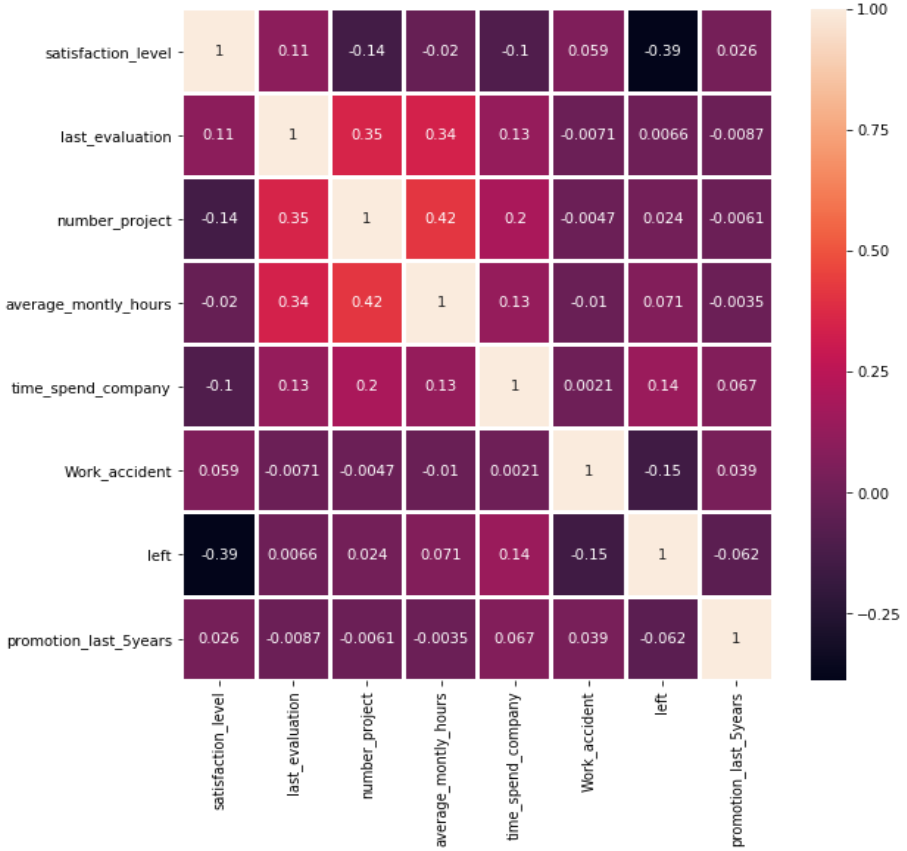
Geramos medidas estatísticas para as variáveis numéricas para verificar existe algo incomum em alguma das colunas.

Por exemplo, colunas cujo desvio padrão, representado pela sigla STD, é igual a 0, possuem o mesmo valor em todas as linhas

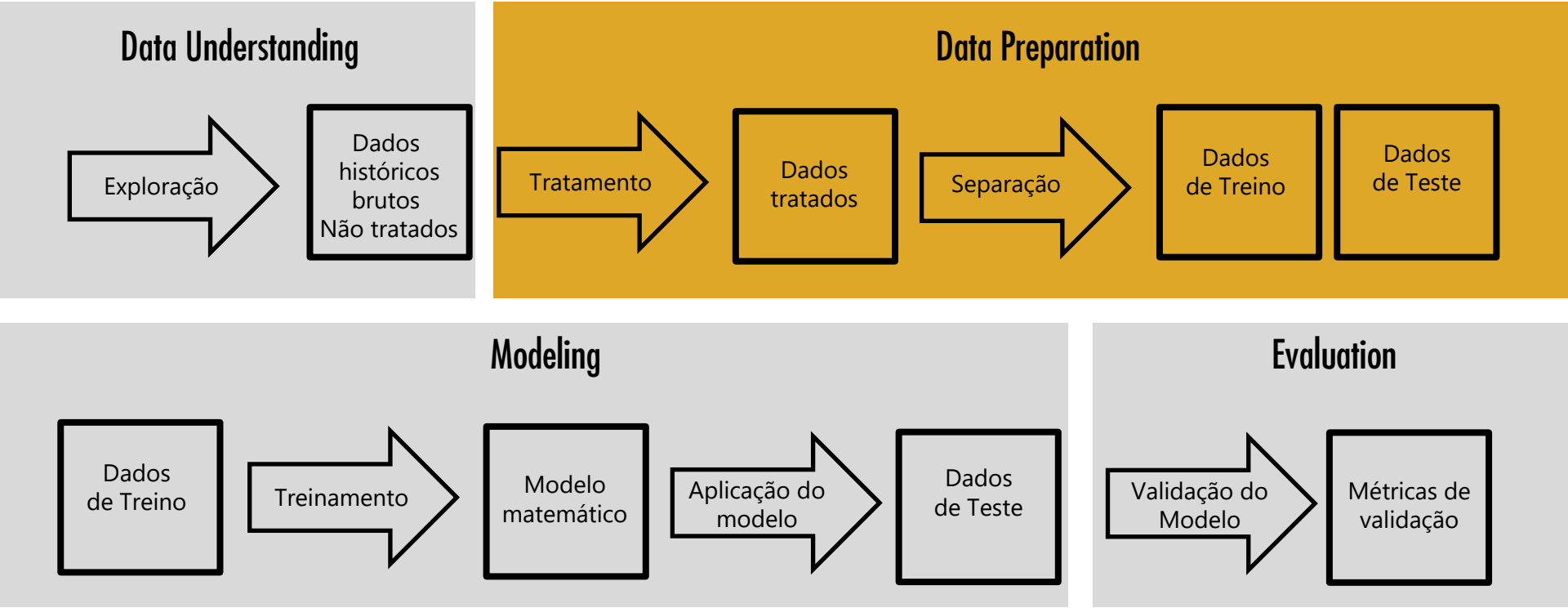
	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years
count	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000
mean	0.612834	0.716102	3.803054	201.050337	3.498233	0.144610	0.238083	0.021268
std	0.248631	0.171169	1.232592	49.943099	1.460136	0.351719	0.425924	0.144281
min	0.090000	0.360000	2.000000	96.000000	2.000000	0.000000	0.000000	0.000000
25%	0.440000	0.560000	3.000000	156.000000	3.000000	0.000000	0.000000	0.000000
50%	0.640000	0.720000	4.000000	200.000000	3.000000	0.000000	0.000000	0.000000
75%	0.820000	0.870000	5.000000	245.000000	4.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	7.000000	310.000000	10.000000	1.000000	1.000000	1.000000

Data Understanding

Nessa etapa também é analisada a correlação entre as variáveis. Alta correlação entre as variáveis pode impactar diretamente o modelo gerado, além de que estamos mantendo muitas vezes colunas que retornam praticamente a mesma informação. A correlação é computada pelo coeficiente de Pearson, que gera uma escala de -1 a 1. -1 significa que duas variáveis são inversamente proporcionais, 1 significa que ambas são proporcionais, e valores próximos a 0 demonstram baixa correlação entra as variáveis



Para os propósitos desta demonstração, iremos abordar as etapas de Data Understanding até Evaluation:



Data Preparation

Não podemos alimentar algoritmos com dados brutos da forma como eles são coletados. Uma série de questões estão envolvidas na preparação dos dados, desde performance até escolha do modelo. Variáveis numéricas de grandezas muito diferentes, outliers, variáveis categóricas que precisam ser quantificadas para que possamos alimentar um algoritmo

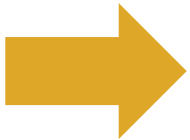
Variável independente	Variável independente	Variável independente	Variável independente	Target, Variável Dependente
A	1	A	100	Y
D		B	122	Y
C	4	B	76	Y
E	3			Y
F	2	A	1300	Y
	2	a	97	Y

que é puramente matemático, eliminar correlações fortes, etc. Essa é a etapa MAIS IMPORTANTE no processo. O tratamento e preparação adequado dos dados é um fator crítico para a eficácia do modelo que será gerado.

Data Preparation - Normalização

Uma das questões que podem impactar determinados algoritmos é a diferença de grandezas entre variáveis numéricas. Por exemplo, um dataset sobre maquinário industrial pode ter variáveis de temperatura medidas entre 0 e 100ºC, tempo de execução na casa dos segundos ou milissegundos, e o peso das máquinas em kilos , com valores entre 1000 e 10000kg. Resolvemos esse problema utilizando técnicas de normalização. Uma delas, utilizada na demonstração, usamos uma fórmula matemática para encaixar todos os valores numéricos das variáveis em uma escala de 0 à 1, onde 0 é o menor valor presente na coluna e 1 é o maior

number_project	average_montly_hours	time_spend_company
2	157	3
5	262	6
7	272	4
5	223	5
2	159	3



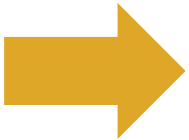
	number_project	average_montly_hours	time_spend_company
0	0.0	0.285047	0.125
1	0.6	0.775701	0.500
2	1.0	0.822430	0.250
3	0.6	0.593458	0.375
4	0.0	0.294393	0.125

Data Preparation – Tratamento de variáveis categóricas e texto

Variáveis categóricas (aquelas onde cada valor – seja ele um texto ou número – representa uma categoria) requerem uma análise e preparação diferente de variáveis não categóricas e por isso vamos explorá-las separadamente das demais.

O ponto importante de mencionar aqui, é que em machine learning, algoritmos de treinamento e modelos gerados são puramente matemáticos, o que significa que qualquer dado que for usado para alimentar esses algoritmos precisa ser de alguma forma quantificado.

Variável independente	Variável independente
A	A
D	B
C	B
E	B
F	A
A	A



Variável independente	Variável independente
X	X
X	X
X	X
X	X
X	X
X	X

Data Preparation – Dummies

No caso de uma coluna com muitos valores diferentes e **sem ordenação lógica**, uma ideia para quantificar é criando **dummies**. Cada valor existênte na coluna é transformado em uma nova coluna. Para cada linha, uma das novas colunas receberá 1 (a coluna com o valor original da linha) e as demais receberão 0

Departments
sales
sales
sales
sales
sales

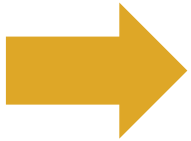


Departments_IT	Departments_RandD	Departments_accounting	Departments_hr	Departments_management	Departments_marketing	Departments_product_mng	Departments_sales	Departments_support	Departments_technical
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	0

Data Preparation – Mapeamento

Já para colunas como a de salário possuem uma **ordem lógica de importância**, já que um salário alto (high) é maior do que um salário médio (medium) que por sua vez é maior do que um salário baixo (low), mesmo que nós não tenhamos o valor exato. Nesse caso podemos quantificar essa coluna mapeando os valores e respeitando a normalização aplicada em colunas numéricas.

Departments	salary
sales	low
sales	medium
sales	medium
sales	low
sales	low



salary
0.0
0.5
0.5
0.0
0.0

Data Preparation – Mapeamento a variável Alvo

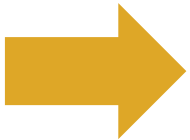
A variável satisfaction_level é uma variável quantitativa contínua (valores decimais). Dentro de aprendizagem supervisionada, temos tarefas de classificação / previsão e regressão. Regressões geram modelos que calculam o valor de Y dado os valores de X, e verificamos o modelo dado o quanto o resultado se afastou o valor real de Y daqueles dados (função de erro). Já classificações e previsões não retornam valores decimais, mas sim classes, rótulos, etc. A ideia dessa demonstração é gerar um classificador, e não uma regressão. Para isso, precisamos fazer com a coluna satisfaction_level um mapeamento. Vamos assumir 3 classes de satisfação: baixa (1), média (2) e alta (3). Nosso modelo deve portanto, classificar a satisfação de um funcionário como baixa, média ou alta. Abaixo temos a função em python, como curiosidade, que realiza esse mapeamento.

```
def mapear_satisfacao(x):
    if (x <= 0.3):
        return 1
    elif (x > 0.3) and (x <= 0.6):
        return 2
    else:
        return 3
```

Data Preparation – Mapeamento a variável Alvo

Aplicando a função na variável alvo:

	satisfaction_level
0	0.38
1	0.80
2	0.11
3	0.72
4	0.37



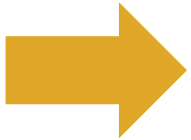
	satisfaction_level
0	2
1	3
2	1
3	3
4	2

Data Preparation – Train Test Split

Por fim, com os dados prontos são separados em X e Y, e depois em dados de teste e dados de treino. A separação dos conjuntos deve respeitar o balanceamento de classes no dataset (esse é um assunto muito importante porém que não será abordada nessa demonstração para manter a simplicidade)

Dataset

Variável independente	Variável independente	Variável independente	Variável independente	Target, Variável Dependente
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y



X Treino

Variável independente	Variável independente	Variável independente	Variável independente
x	x	x	x
x	x	x	x
x	x	x	x
x	x	x	x

Y Treino

Target, Variável Dependente
Y
Y
Y
Y

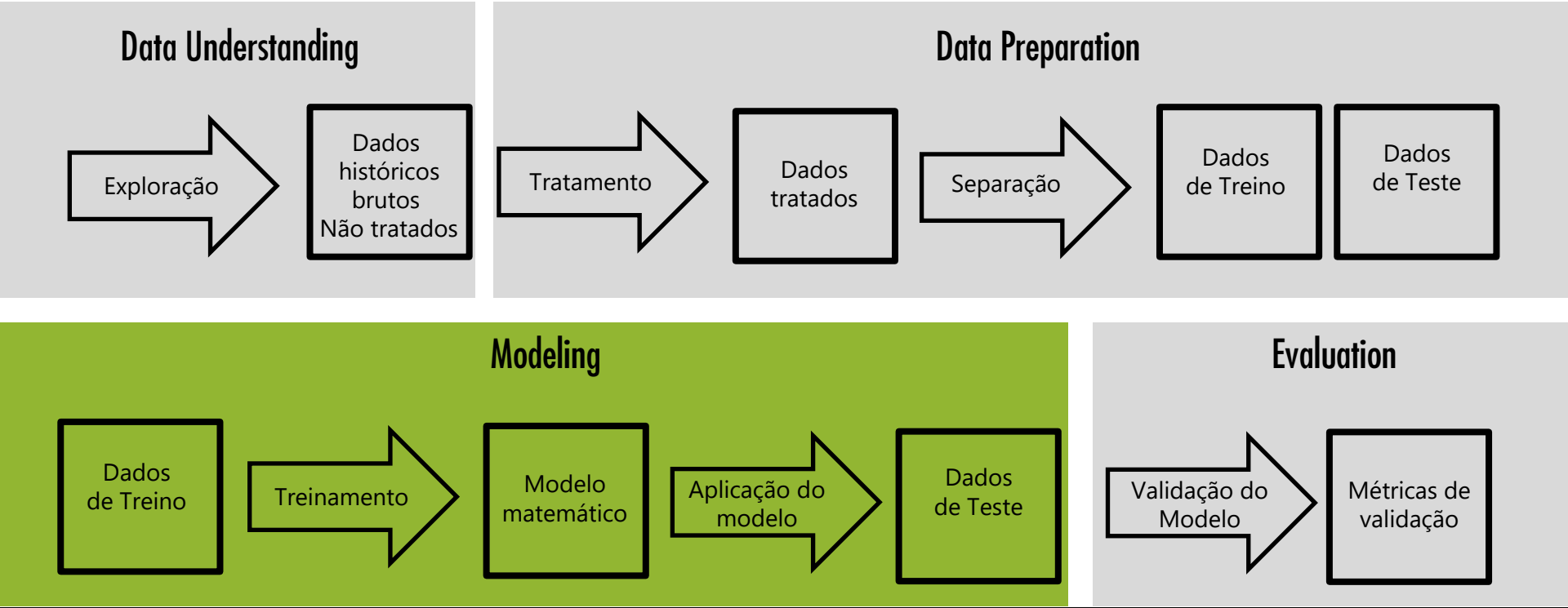
X teste

Variável independente	Variável independente	Variável independente	Variável independente
x	x	x	x
x	x	x	x

Y Teste

Target, Variável Dependente
Y
Y

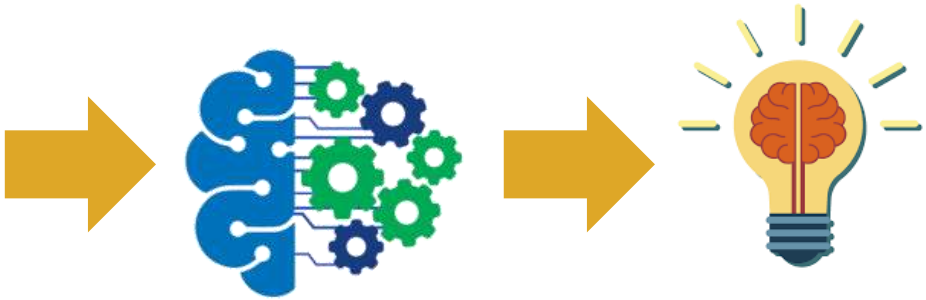
Para os propósitos desta demonstração, iremos abordar as etapas de Data Understanding até Evaluation:



Modeling

Agora que temos os dados devidamente tratados e separados, podemos realizar a fase de aprendizagem (onde de fato entra IA nessa história). Uma coisa interessante de se reparar, é que embora exista toda a fama envolta de IA e Machine Learning, essa etapa é completamente dependente das etapas anteriores, e costuma ser menor do que a etapa de preparação.

Variável independente	Variável independente	Variável independente	Variável independente	Target, Variável Dependente
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y
x	x	x	x	Y



Modeling

Criamos uma função no python que realiza o processo de treino e teste por completo, realizando as seguintes etapas:

1) Recebe o modelo e os dados já separados
2) Treina o algoritmo com os dados de treino
3) Testa o modelo nos dados de treino
4) Computa as métricas de avaliação de treino
5) Testa o modelo nos dados de teste
6) Computa as métricas de avaliação de teste

```
def fit_predict(modelo, X_train, y_train, X_test, y_test):
    resultados = {}

    # processo de treino
    start = time()
    modelo.fit(X_train, y_train)
    end = time()
    resultados['tempo_treino'] = end - start

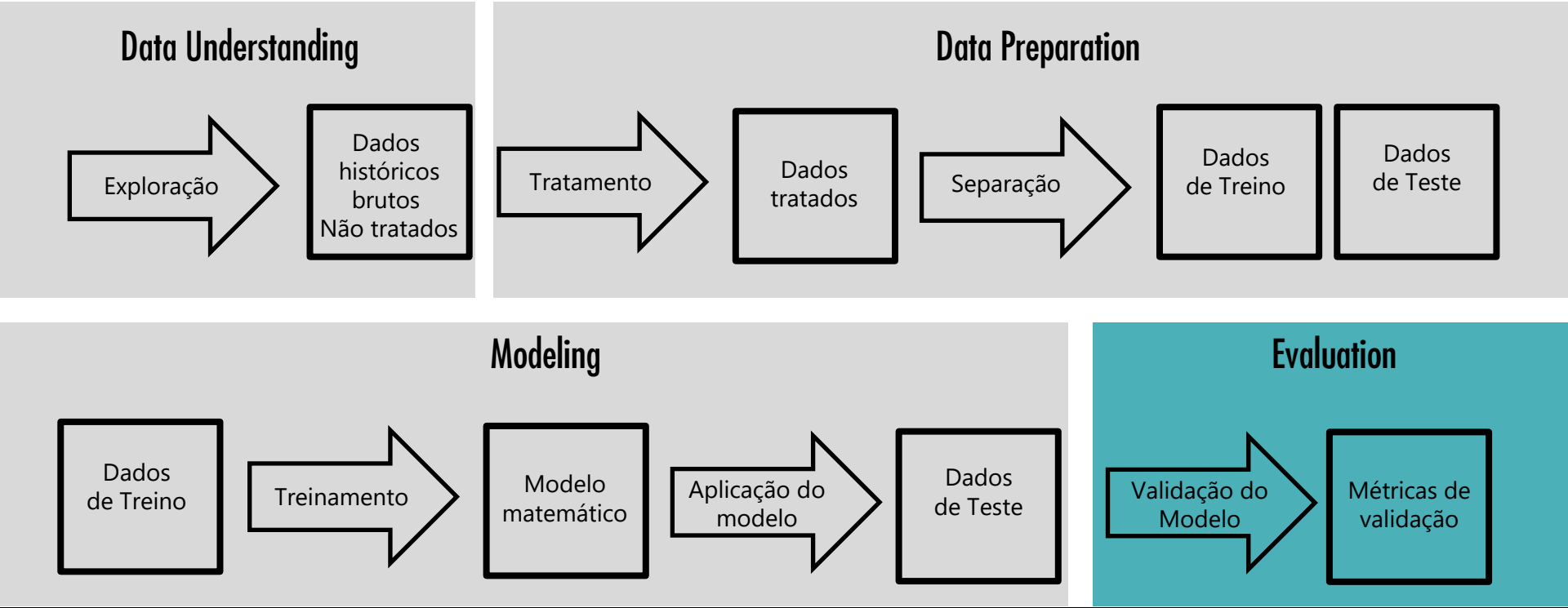
    # processo de teste
    start = time()
    previsoes_treino = modelo.predict(X_train)
    previsoes_teste = modelo.predict(X_test)
    end = time()

    # computa as métricas de treino e teste para avaliação
    resultados['tempo_previsão'] = end - start
    resultados['accuracy_teste'] = accuracy_score(y_test, previsoes_teste)
    resultados['accuracy_treino'] = accuracy_score(y_train, previsoes_treino)
    resultados['precision_teste'] =  precision_score(y_test, previsoes_teste, average='macro')
    resultados['precision_treino'] =  precision_score(y_train, previsoes_treino,
average='macro')
    resultados['recall_teste'] =  recall_score(y_test, previsoes_teste, average='macro')
    resultados['recall_treino'] =  recall_score(y_train, previsoes_treino, average='macro')
    resultados['fscore_teste'] = fbeta_score(y_test, previsoes_teste, beta=0.5,
average='macro')
    resultados['fscore_treino'] = fbeta_score(y_train, previsoes_treino, beta=0.5,
average='macro')

    # Successo
    print('Processo de modelagem com %s finalizado' % modelo.__class__.__name__)

    # Retorna os resultados e as previsões
    return pd.Series(resultados), previsoes_treino, previsoes_teste
```

Para os propósitos desta demonstração, iremos abordar as etapas de Data Understanding até Evaluation:



Evaluation

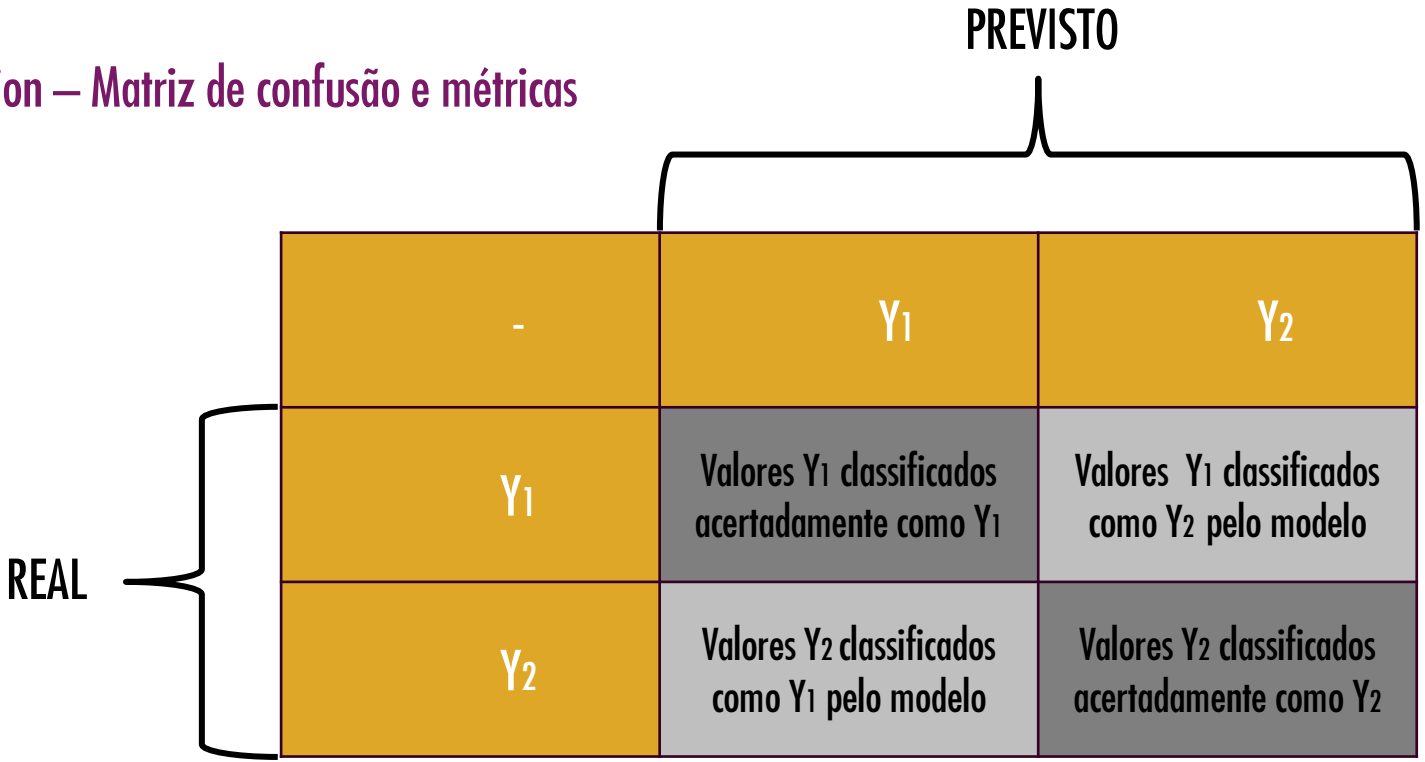
Todo algoritmo de machine learning pode ser validado. Dependendo do algoritmo e do tipo de aprendizagem, diferentes métricas são utilizadas. No caso de aprendizagem supervisionada, onde temos uma coluna Y estabelecida, os algoritmos geralmente treinam usando como referência uma função de erro aplicada encima de Y.



Basicamente, se sabemos qual a classificação real de cada linha do dataset, podemos ajustar o modelo até que ele erre menos ao classificar essas linhas. Em outras palavras, a função de erro é computada comparando a classificação do modelo com a classificação real. Essa etapa acontece durante a criação do modelo matemático, e também na validação do modelo matemático. Note que essa é uma generalização, dependendo do algoritmo, o processo pode ser completamente diferente.

live university

Evaluation – Matriz de confusão e métricas



Accuracy = Previsões Corretas / Total de Previsões

Evaluation – Notação comum

Na literatura, vemos muito termos como *Positivo verdadeiro*, *Falso positivo*, etc, especialmente em casos de classificação binária (duas classes, 0 e 1)

-	Negativo	Positivo
Negativo	Negativo Verdadeiro	Falso Positivo
Positivo	Falso Negativo	Positivo Verdadeiro

Precision, Recall e Fbeta Score

Precision

-	Negativo	Positivo
Negativo	Negativo Verdadeiro	Falso Positivo
Positivo	Falso Negativo	Positivo Verdadeiro

Precision = Positivo Verdadeiro/ (Positivo Verdadeiro + Falso Positivo)

Precision, Recall e Fbeta Score

-	Negativo	Positivo
Negativo	Negativo Verdadeiro	Falso Positivo
Positivo	Falso Negativo	Positivo Verdadeiro

Recall

Recall = Positivo Verdadeiro/ (Positivo Verdadeiro + Falso Negativo)

Precision, Recall e Fbeta Score

Precision

-	Negativo	Positivo
Negativo	Negativo Verdadeiro	Falso Positivo
Positivo	Falso Negativo	Positivo Verdadeiro

Recall

$$Fbeta = 2 * \frac{(Precision * Recall)}{(Precision + Recall)}$$

Evaluation – Resultados dos algoritmos

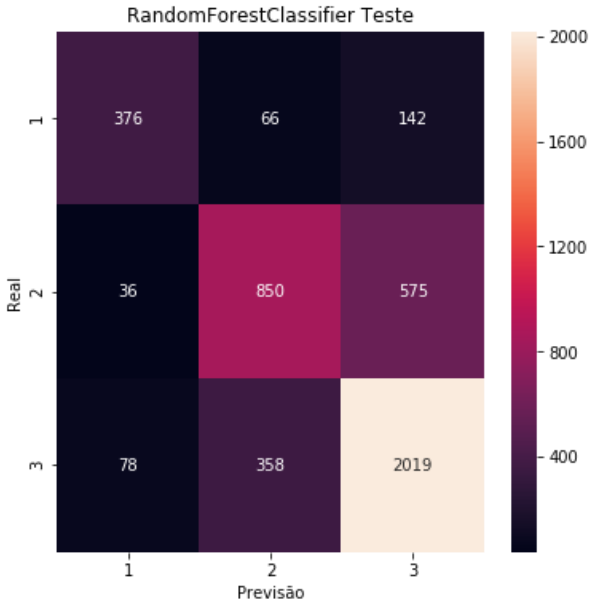
O RandomForest foi o modelo que melhor performou de uma forma geral, mas algo muito interessante de ser observado é a grande queda de performance que esse modelo tem quando comparamos a previsão feita encima dos dados de treino e os dados de teste (muito mais acentuada do que os outros modelos).

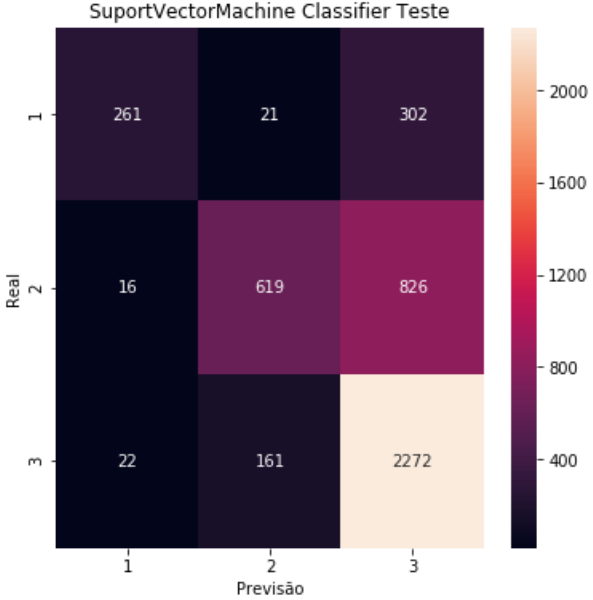
Overfitting (sobreajuste) acontece no momento de treinar o algoritmo, já que o modelo matemático é gerado encima dos dados de treino. O modelo fica muito bem ajustado aos dados de treino, de certa forma, enviesado aqueles dados. Todo algoritmo apresenta algum grau de overfitting, porém, árvores de decisão são famosas por sofrerem bastante com isso.

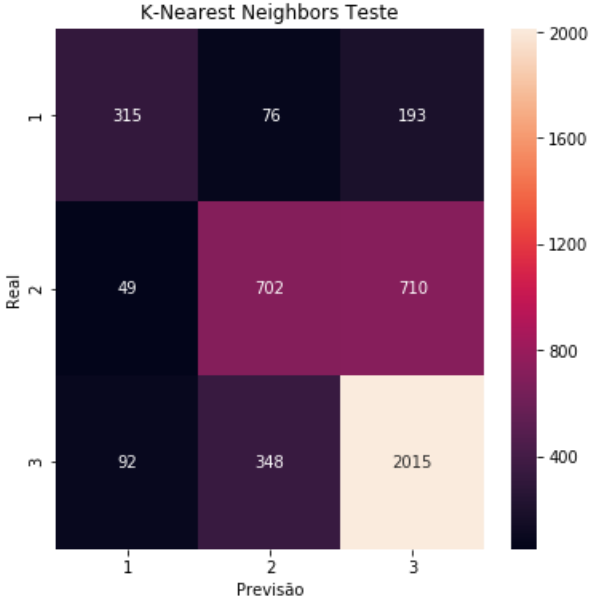
	KNeighborsClassifier	RandomForestClassifier	SVC
accuracy_teste	0.673778	0.721111	0.700444
accuracy_treino	0.786837	0.988189	0.721688
fscore_teste	0.651892	0.713523	0.701388
fscore_treino	0.780316	0.988001	0.728069
precision_teste	0.668259	0.724158	0.771310
precision_treino	0.794845	0.988389	0.798420
recall_teste	0.613550	0.682677	0.598686
recall_treino	0.740756	0.986468	0.619748
tempo_previsão	1.175925	0.033908	3.400038
tempo_treino	0.033909	0.112772	4.000132

live university

Evaluation - Matriz de confusão - Comparação







Copyright

© Live University. Este trabalho, incluindo todos os modelos, slides e textos estão sob a licença Creative Commons Atribuição-NãoComercial-CompartilhaIgual 4.0 Internacional.
Para ver uma cópia desta licença, visite http://creativecommons.org/licenses/by-nc-sa/4.0/
Você pode utilizar e distribuir esse material desde que a fonte "Live University" seja mencionada.
Autorizações adicionais podem ser concedidas no âmbito desta licença pelo e-mail abaixo:
alex.leite@Live University.org.br

© Live University . This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. To view a copy of this license, visit
http://creativecommons.org/licenses/by-nc-sa/4.0/
You must give appropriate credit to Live University, provide a link to the license, and indicate if changes were made. You may not use the material for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

live university